

OSSP (Operating Systems And Systems Programming) - 25CS2104E

PRACTICAL 4

Objective: Create multiple child processes, synchronize them using wait() and waitpid(), compare both functions, demonstrate a zombie, inspect it, and eliminate it using proper synchronization.

Step 1 — Open Ubuntu Terminal

Press Ctrl + Alt + T.

Ctrl + Alt + T

You should see something similar to:

Ubuntu terminal opens.

Step 2 — Create the Practical Folder

Create and enter the folder.

```
mkdir -p ~/osspractical4
cd ~/osspractical4
pwd
```

Step 3 — Create the Multiple-Child Program

Open the source file.

nano multiple_children.c

You should see something similar to:

GNU nano opens.

Paste this program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t c1, c2, c3, finished;

    c1 = fork();
    if (c1 == 0) {
        printf("Child 1: PID=%d PPID=%d\n", getpid(), getppid());
        sleep(2);
        exit(10);
    }

    c2 = fork();
    if (c2 == 0) {
        printf("Child 2: PID=%d PPID=%d\n", getpid(), getppid());
        sleep(4);
        exit(20);
    }
}
```

```

}

c3 = fork();
if (c3 == 0) {
    printf("Child 3: PID=%d PPID=%d\n", getpid(), getppid());
    sleep(6);
    exit(30);
}

printf("Parent PID=%d\n", getpid());
printf("Children: %d %d %d\n", c1, c2, c3);

finished = wait(NULL);
printf("wait() collected child PID=%d\n", finished);

finished = wait(NULL);
printf("wait() collected child PID=%d\n", finished);

finished = wait(NULL);
printf("wait() collected child PID=%d\n", finished);

printf("Parent: all children completed\n");
return 0;
}

```

Save: Ctrl + O → Enter → Ctrl + X

Step 4 — Compile

Compile the program.

```
gcc multiple_children.c -o multiple_children
```

Step 5 — Run the Program

Run and observe multiple children.

```
./multiple_children
```

The exact order can vary because process scheduling is controlled by Linux.

Step 6 — Understand wait()

wait() waits for any child that has terminated.

```
finished = wait(NULL);
```

You should see something similar to:

When a child terminates, wait() returns the PID of a terminated child.
If several children exist, the parent does not select a particular child with wait().

Step 7 — Create the waitpid() Program

Create a second program to demonstrate targeted synchronization.

```
nano waitpid_demo.c
```

You should see something similar to:

GNU nano opens.

Paste:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t c1, c2;
    int status;

    c1 = fork();
    if (c1 == 0) {
        printf("Child 1 PID=%d\n", getpid());
        sleep(2);
        exit(10);
    }

    c2 = fork();
    if (c2 == 0) {
        printf("Child 2 PID=%d\n", getpid());
        sleep(5);
        exit(20);
    }

    printf("Parent waiting for Child 2: %d\n", c2);

    waitpid(c2, &status, 0);

    if (WIFEXITED(status))
        printf("Child 2 exit status=%d\n", WEXITSTATUS(status));

    waitpid(c1, &status, 0);

    printf("Parent: both children collected\n");
    return 0;
}
```

Step 8 — Compile and Run waitpid()

Compile and execute the targeted wait program.

```
gcc waitpid_demo.c -o waitpid_demo
./waitpid_demo
```

Step 9 — Compare wait() and waitpid()

Record the difference between the two synchronization calls.

You should see something similar to:

wait() → waits for any child; simpler; returns a terminated child's PID.
waitpid(pid,...) → waits for a selected child; provides more control and status information.
wait() is useful when any completed child is acceptable.
waitpid() is useful when a specific child must be synchronized.

Step 10 — Create a Zombie Process

A zombie is a child that has terminated but whose parent has not yet collected its exit status.

```
nano zombie.c
```

You should see something similar to:

GNU nano opens.

Paste this program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child PID=%d: exiting now\n", getpid());
        exit(0);
    }

    printf("Parent PID=%d\n", getpid());
    printf("Child PID=%d\n", pid);
    printf("Parent sleeping; child is now a zombie\n");

    sleep(20);

    printf("Parent exiting\n");
    return 0;
}
```

Step 11 — Compile and Run Zombie Program

Run it in the background so you can inspect the process table.

```
gcc zombie.c -o zombie
./zombie &
```

Step 12 — Find the Zombie

Inspect PID, PPID and state.

```
ps -o pid,ppid,stat,cmd -p 13200,13201
```

Z means zombie. Use your actual PIDs.

Step 13 — Understand the Zombie Flow

Record why the zombie appears.

Child exits → parent does not call wait() → child becomes zombie → parent eventually collects it or exits.

You should see something similar to:

Zombie state = Z
The child is already terminated, but its process-table entry remains until the parent collects its status.

Step 14 — Fix the Zombie

Modify the parent to wait for the child.

```
#include <sys/wait.h>
wait(NULL);
```

You should see something similar to:

The parent waits for the child, collects its exit status, and removes the zombie entry.

Step 15 — Compile and Run the Fixed Version

Compile and execute again.

```
gcc zombie.c -o zombie  
./zombie
```

Step 16 — Verify Zombie Removal

Check the process table after the child is collected.

```
ps -o pid,ppid,stat,cmd | grep zombie
```

No zombie entry should remain after proper synchronization.

FINAL RESULT: Multiple children are synchronized, `wait()` and `waitpid()` are compared, a zombie process is demonstrated and identified with `ps`, and the zombie is removed using `wait()`.

Source: OSSP_Practical.docx — Practical 4.